

Relatório do Trabalho Prático 3 - MAC0352

Minipilha

Nomes:

Lauferrando Souza Dias - 11222947

Rafael Lopes Santana- 14604420

João Pedro de Toledo - 10724177

Erick Henrique Imai Rodrigues - 7577880

1. Arquitetura

ED: fila de mensagens (protegida por mutex); a separação entre os clientes fica mais parecida com o EP2 (onde o servidor teria o papel parecido com o manager).

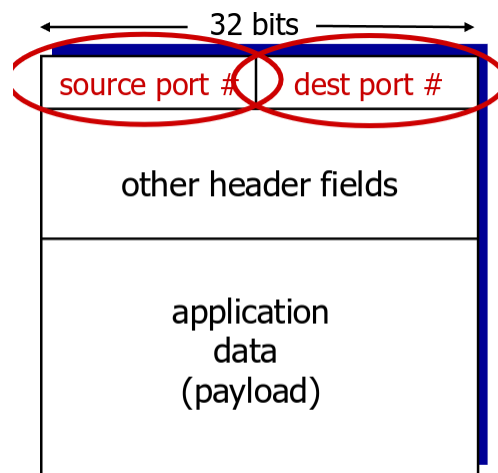
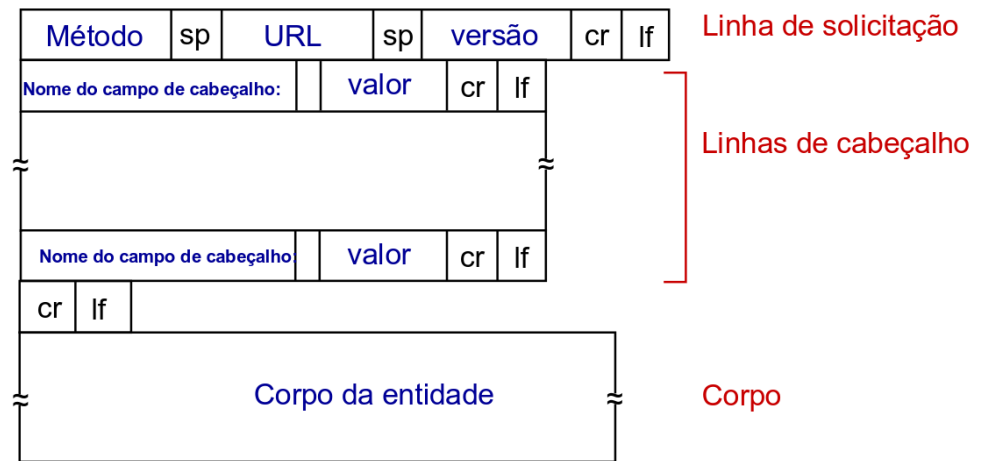
1.1 Base da Arquitetura do Sistema

O sistema (chat simples peer-to-peer) foi organizado como uma mini-pilha de protocolos em quatro camadas: aplicação, transporte, rede e enlace. A aplicação implementa a lógica do chat. A camada de transporte fornece entrega confiável com controle de sequência, ACK e retransmissão. A camada de rede encapsula segmentos em pacotes com endereçamento lógico. A camada de enlace simula o meio físico por meio de frames enviados a todos os endereços listados no arquivo `hosts`, reproduzindo o comportamento de um hub compartilhado.

A execução final ocorre sobre TCP real, usado apenas como suporte de comunicação entre processos locais. Sobre esse suporte, a pilha própria simula a transmissão, a perda, o atraso e a entrega entre nós lógicos.

Slides de aula em que a formatação de mensagens e segmentos se baseou:

Mensagem de solicitação HTTP: formato geral



TCP/UDP segment format

No código:

```

/*
 * Segmento da camada de transporte.
 *
 * Cabeçalho serializado:
 *   sourcePort(4) | destinationPort(4) | seqNumber(4) | ackNumber(4)
 *   | checksum(4) | flags(4) | data_size(4)
 * = 7 × 4 = 28 bytes de cabeçalho.
 */
You, 5 days ago | 2 authors (You and one other)
struct Segment {
    int sourcePort;
    int destinationPort;
    int seqNumber;
    int ackNumber;
    // soma simples dos bytes do payload (módulo 2^32)
    int checksum;
    int flags;
    int data_size;
    char data[SEG_MAX_DATA];
};

```

1.2 Camada de Enlace

Introduz ruído aleatório nas perdas de pacotes.

1.3 Camada de Rede

Faz um IP simulado, tenta enviar para esse IP.

1.4 Camada de Transporte - TCP

Implementar os ACKs com timeout, SYN (sem FIN).

1.5 Camada de Aplicação (chat simples)

Faz com que o servidor repasse mensagem entre clientes.

2. Descrição das responsabilidades de cada camada

Camada de aplicação

Responsável por representar mensagens do chat, realizar o handshake de sessão e tratar envio e recebimento de texto entre os usuários.

Camada de transporte

Responsável por oferecer entrega confiável em modo stop-and-wait. Implementa número de sequência, ACK, timeout, retransmissão e verificação de integridade por checksum.

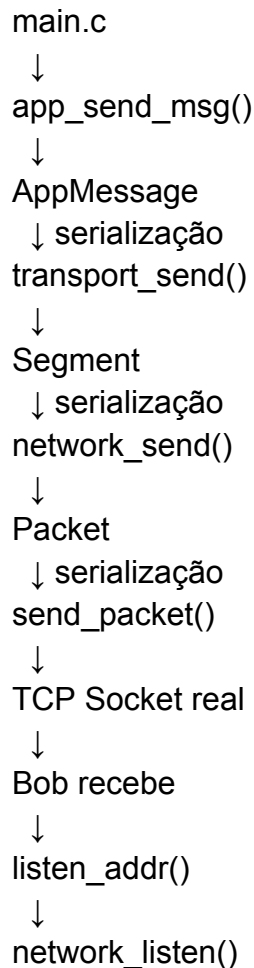
Camada de rede

Responsável por encapsular o segmento em um pacote com endereços de origem e destino, funcionando como uma abstração simples de IP.

Camada de enlace

Responsável por simular o envio no meio compartilhado. O frame é enviado para todos os endereços do arquivo `hosts`, e cada receptor filtra o que não lhe pertence.

Fluxo de Mensagem de um usuário a outro:



↓
transport_recv()
↓
app_recv_loop()
↓
Tela

Estabelecimento de conexão simples:

```
→ EP3 git:(main) ./client 1 2 1001 2001 0.0.0.0 7001 Alice --hosts hosts_alice
[config] loss=0.00 delay=500 offset=0 reliable=1
```

MiniIRC Chat Client

```

  Usuário : Alice
  IP lógico: 1 → 2
  Escutando: 0.0.0.0:7001

[transport] enviando seq=0 tentativa=1
[transport] ACK recebido para seq=0
[sessão] CONNECT enviado como 'Alice'
[transport] segmento seq=0 recebido (39 bytes)
[sessão] conectado! Peer: 'Bob'

Digite uma mensagem e pressione Enter. /help para ajuda.

> [transport] segmento seq=1 recebido (42 bytes)

Bob: ola
> oi
[transport] enviando seq=1 tentativa=1
[transport] timeout aguardando ACK seq=1
[transport] enviando seq=1 tentativa=2
Alice: oi
```

```
→ EP3 git:(main) ./client 2 1 2001 1001 0.0.0.0 7002 Bob --passive --hosts hosts_bob
[config] loss=0.00 delay=500 offset=0 reliable=1

MiniIRC Chat Client

Usuário : Bob
IP lógico: 2 → 1
Escutando: 0.0.0.0:7002

[sessão] aguardando conexão de 1...

Digite uma mensagem e pressione Enter. /help para ajuda.

> [transport] segmento seq=0 recebido (44 bytes)

*** 'Alice' conectou ***
> [transport] enviando seq=0 tentativa=1
[transport] ACK recebido para seq=0
ola
[transport] enviando seq=1 tentativa=1
[transport] timeout aguardando ACK seq=1
[transport] enviando seq=1 tentativa=2
Bob: ola
> [transport] segmento seq=1 recebido (41 bytes)

Alice: oi
```

3. Interfaces entre camadas

A interface entre aplicação e transporte é a passagem de dados de chat, já encapsulados em mensagens da aplicação. A interface entre transporte e rede é o segmento, contendo portas, sequência, ACK, checksum, flags e dados. A interface entre rede e enlace é o pacote, com cabeçalho de rede e payload. A camada de enlace, por sua vez, converte o pacote em frame e o distribui aos processos participantes.

Cada camada trata apenas o formato imediatamente inferior, evitando dependência direta entre componentes não adjacentes.

4. Formato dos cabeçalhos e encapsulamento

Aplicação

A mensagem da aplicação possui tipo, nome de usuário, identificador da mensagem, tamanho do payload e payload em si.

Transporte

O segmento contém portas de origem e destino, número de sequência, número de ACK, checksum, flags e tamanho do dado. O encapsulamento adiciona o cabeçalho de transporte ao conteúdo da aplicação.

Rede

O pacote contém tamanho do cabeçalho, tamanho total, endereço de origem, endereço de destino e o payload da camada inferior.

Enlace

O frame contém endereço de origem, endereço de destino e dados já encapsulados. Como a rede é simulada em broadcast local, todos os processos recebem o frame, mas apenas o destinatário correto o aceita.

5. Decisões de projeto

Uma decisão central foi adotar stop-and-wait, pois simplifica a implementação da confiabilidade e torna os efeitos de perda e atraso mais fáceis de observar. Outra decisão foi manter a simulação de enlace baseada em envio para todos os hosts do arquivo `hosts`, o que representa um meio compartilhado sem necessidade de roteamento real.

Também foi adotado um checksum simples para reduzir complexidade e manter o foco na lógica da pilha. O uso de portas lógicas permite multiplexação entre fluxos de aplicação, mesmo com um único processo de transporte.

6. Metodologia experimental

Os experimentos foram organizados variando parâmetros de configuração do meio, especialmente taxa de perda, atraso médio e modo confiável ou não confiável. A observação foi feita por meio de logs gerados pelo próprio sistema, permitindo analisar tentativas de envio, timeouts, ACKs recebidos e mensagens efetivamente entregues.

A análise do overhead foi feita a partir do tamanho dos cabeçalhos definidos em cada camada, comparando o custo de encapsulamento com o tamanho útil dos dados transmitidos.

Devido a problemas nos scripts de teste, a entrega está com os mesmos ausentes no diretório do projeto.

7. Resultados e análise

Aumento de perda tende a elevar o número de retransmissões e, em casos extremos, pode impedir a abertura da sessão ou a entrega final das mensagens. Aumento de atraso afeta diretamente o tempo total de comunicação, mesmo quando não há perda, porque cada ciclo de envio aguarda confirmação.

No modo confiável, a pilha busca preservar a entrega ao custo de retransmissões e maior latência. No modo não confiável, o custo é menor, porém a taxa de entrega cai conforme a perda aumenta. Isso evidencia o trade-off entre desempenho e confiabilidade.

O overhead de encapsulamento é relevante, principalmente para mensagens curtas, pois os cabeçalhos somados das camadas representam uma parcela expressiva do total transmitido.

8. Limitações e melhorias futuras

A implementação atual é adequada para um cenário didático e ponto a ponto, mas ainda possui limitações. O controle de sessão é simples e depende da interação entre dois processos. O checksum é básico e não substitui mecanismos mais robustos de integridade. O comportamento da camada de enlace é artificial e não contempla endereçamento, colisão ou fila real de rede.

Como melhorias futuras, seria possível adicionar múltiplas sessões simultâneas, melhor tratamento de encerramento de conexão, cálculo de estatísticas automáticas e uma simulação mais realista do enlace e da rede. Também seria interessante ampliar o protocolo da aplicação para comandos e estados mais próximos de um chat real.